Kapnet Over Bitcoin

Volume III

Canonical Schemas, Opcode Semantics, Knot Claims, Namespace Grammar, Relay Contracts, and Node Workflows

This volume narrows the broader doctrine into an exacting conceptual specification. It defines packet families, canonical field semantics, a concrete opcode core, knot-claim structure, namespace grammar, relay contract templates, replay rules, and operator workflows. It is still a conceptual standard rather than a final byte-perfect RFC, but it is written to behave like one.

March 24, 2026

Contents

Abstract **2**

1 Scope, Audience, and Conformance Language **3**

1.1 Scope . . . 3

1.2 Audience . . . 3

1.3 Normative language . . . 3

2 Canonical Object Model **4**

2.1 Primary object families . . . 4

2.2 Object identifiers . . . 4

2.3 Stable vs. evolving identity . . . 5

3 Canonical Packet Schema **5**

3.1 Packet philosophy . . . 5

3.2 Common packet envelope . . . 6

3.3 Packet families . . . 6

3.4 Canonical serialization rules . . . 7

3.5 Size classes . . . 7

4 Namespace Grammar and Destination Resolution **8**

4.1 Design goals . . . 8

4.2 ABNF-like grammar . . . 8

4.3 Examples . . . 9

4.4 Resolution order . . . 9

4.5 Compound prefixes . . . 9

5 Identity, Delegation, and Representation **9**

5.1 Author modes . . . 9

5.2 Capability scopes . . . 10

5.3 Delegation packets . . . 10

6 Typed Bodies for Core Packet Families **11**

6.1 `SUBMIT` body . . . 11

6.2 `CREATE_OBJ` body . . . 11

6.3 `TRANSITION` body . . . 11

6.4 `ATTEST` body . . . 12

6.5 `CONTRACT` body . . . 12

6.6 `RECEIPT` body . . . 12

6.7 `FINGERPRINT` body . . . 13

7 Constitutional Opcode Core **13**

7.1 Purpose . . . 13

7.2 Execution model . . . 13

7.3 State transition categories . . . 13

7.4 Opcode table . . . 14

7.5 Execution bounds . . . 15

7.6 Failure semantics . . . 15

8 State Transition Rules by Object Class **16**

8.1 Room transitions . . . 16

8.2 Guild transitions . . . 16

8.3 DAO transitions . . . 17

8.4 Provider transitions . . . 17

9 Knot Claim Structure **17**

9.1 Role of knot claims . . . 17

9.2 Knot claim body . . . 17

9.3 Required invariants . . . 18

9.4 Ordering rules . . . 18

9.5 Competing claims . . . 19

9.6 Knot acceptance workflow . . . 19

10 Fingerprints and Many-Tip State **19**

10.1 Purpose of fingerprints . . . 19

10.2 Fingerprint fields . . . 19

10.3 Cross-tip semantics . . . 20

11 Relay Service Classes **20**

11.1 Why service classes are explicit . . . 20

11.2 Core classes . . . 20

11.3 Admission policy structure . . . 20

12 Relay Contract Templates **21**

12.1 Template philosophy . . . 21

12.2 Template R1: Basic relay admission . . . 21

12.3 Template R2: Retention contract . . . 21

12.4 Template R3: Archive pinning . . . 21

12.5 Template R4: Knot aggregation . . . 22

12.6 Template R5: Indexing contract . . . 22

13 Replay Pipeline **22**

13.1 Pipeline stages . . . 22

13.2 Replay pseudocode . . . 22

13.3 Ordering sources . . . 23

14 Receipts **23**

14.1 Why receipts matter . . . 23

14.2 Receipt taxonomy . . . 23

14.3 Receipt reason codes . . . 24

15 Node Workflows **24**

15.1 Why workflows belong in the spec . . . 24

15.2 Workflow W1: Join a guild . . . 24

15.3 Workflow W2: Submit to a room . . . 25

15.4 Workflow W3: Amend a DAO constitution . . . 25
15.5 Workflow W4: Negotiate a relay contract . . . 26
15.6 Workflow W5: Propose a knot . . . 26

16 Node Interface Surfaces 26

16.1 Core operator panes . . . 26
16.2 Minimal screen layouts . . . 27
16.3 Operator alarms . . . 27

17 Worked Examples 28

17.1 Example 1: Editorial room post . . . 28
17.2 Example 2: Guild membership admission . . . 28
17.3 Example 3: Knot claim over publication batch . . . 28

18 Security and Failure Surfaces 29

18.1 Replay ambiguity . . . 29
18.2 Delegation leakage . . . 29
18.3 Namespace capture . . . 29
18.4 Provider overcentralization . . . 29
18.5 Knot ossification or manipulation . . . 29

19 Application and Use-Case Mapping 29

19.1 Why map applications in a technical volume . . . 29
19.2 Editorial and publishing systems . . . 30
19.3 Research and scientific guilds . . . 30
19.4 Firms and cooperatives . . . 30
19.5 Media networks and investigative collaboration . . . 30
19.6 Open policy and municipal governance . . . 30
19.7 Cross-tip bridge communities . . . 30
19.8 Agentic organizations . . . 30

20 Implementation Sequence 31

20.1 Phase 1: packet and object registry 31
20.2 Phase 2: replay engine 31
20.3 Phase 3: providers and contracts 31
20.4 Phase 4: knots and many-tip awareness 32
20.5 Phase 5: operator cockpit 32

21 Conclusion **32**

A Quick Reference: Common Failure Codes **33**

B Quick Reference: Capability Examples **33**

C One-Page Build Checklist **33**

Abstract

This document is the narrowing pass. Prior volumes established doctrine, visual grammar, institutional ontology, relay economics, and the many-tip worldview. This volume attempts to specify the exact middle layer with enough precision that implementation choices become constrained rather than merely inspired. It introduces a canonical packet model, a typed object system for rooms, guilds, DAOs, federations, archives, and providers, a compact but expressive opcode core for constitutional state transitions, a knot-claim envelope, a namespace grammar, relay contract templates, fingerprint structures, replay rules, and operator workflows.

The animating principle is unchanged: Bitcoin remains the hegemonic base chronology and Kapnet remains subordinate epi-consensus. What changes here is granularity. Instead of talking only about submissions in the abstract, this document defines common header fields, object identifiers, signature envelopes, capability scopes, destination resolution, canonical serialization constraints, state transition preconditions, failure rules, checkpoint semantics, and contract life cycles. Instead of speaking only about relays as a market, it enumerates service classes, admissibility gates, receipts, and retention or publication contracts. Instead of treating a node cockpit as a metaphor, it decomposes the panes, counters, and workflow loops a serious operator would actually need.

The goal is not to prematurely freeze one immutable protocol. The goal is to make vagueness expensive. A system like Kapnet can only remain intellectually coherent if every major term eventually bottoms out in packets, rules, transitions, and operator-visible states. Volume III is that descent.

1. Scope, Audience, and Conformance Language

1.1. Scope

This volume addresses six concrete deliverables:

1. a canonical packet schema and serialization discipline,
2. an opcode table and transition semantics for constitutional state,
3. a structured knot claim envelope and replay relation,
4. a namespace grammar with destination resolution rules,
5. relay contract templates and service classes,
6. node workflows and interface surfaces.

It does not attempt to settle every cryptographic primitive, fee formula, or storage backend. It instead fixes the structure within which those choices must fit.

1.2. Audience

The intended readers are:

- protocol designers,
- node implementers,
- relay and provider operators,
- DAO or guild constitution designers,
- interface designers building serious operator tooling,
- researchers attempting to reason about replay, plurality, and civil coordination over Bitcoin time.

1.3. Normative language

The key words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** in this document indicate the strength of a rule. In particular:

- **MUST** means a conforming implementation cannot ignore the requirement without breaking interoperability or doctrinal boundaries.

- **SHOULD** means there may be reasons to deviate, but deviations require clear justification.
- **MAY** means the feature is optional or implementation-specific.

Axiom 1 (Subordination Axiom)**.** *No Kapnet rule may invalidate an otherwise valid base-chain history.*

Axiom 2 (Bounded Replay Axiom)**.** *Every accepted Kapnet state transition must be replayable in bounded time and bounded resource space under the rules of the relevant execution surface.*

2. Canonical Object Model

2.1. Primary object families

All Kapnet activity is modeled as typed objects that can appear either as durable state objects or as event packets that create, mutate, attest to, or checkpoint those state objects.

Object	Meaning	Typical lifetime
Submission	Atomic civil act, human or machine interpretable	event-scoped but archivable
Room	Bounded discussion and rule surface	durable
Guild	Durable institution containing members and rooms	durable
DAO	Executable constitutional object	durable
Federation	Meta-container binding several DAOs or guilds	durable
Archive	Addressable memory object, publication corpus, or snapshot set	durable
Provider	Service actor exposing relay, archive, index, or compute capabilities	durable
Fingerprint	Compact state digest for a tip or domain	refreshable durable record
Braid region	Partially ordered concurrency window	transient but checkpointable
Knot claim	Checkpoint proposal over a braid region or replay window	event-scoped and archivable
Receipt	Service acknowledgment or contractual proof	event-scoped and archivable
Contract	Service agreement for relay, retention, publication, attestation, or indexing	durable or time-bounded

2.2. Object identifiers

Every durable object and every event packet **MUST** carry or derive a stable object identifier.

Definition 1 (Kapnet Object Identifier)**.** *A Kapnet object identifier is a typed identifier of the form*

kind:network:namespace:hash

where ***kind*** *identifies the object family,* ***network*** *identifies the relevant base or overlay domain,* ***namespace*** *situates the object within a visible route space, and* ***hash*** *is a canonical digest of the defining payload or state root.*

Illustrative forms:

- `room:BTC:blz.editorial:ab12...`
- `dao:BTC:kap.core:5c44...`
- `submission:BTC:blz.history:9e01...`
- `knot:CTB:newswire:41ff...`

2.3. Stable vs. evolving identity

A Kapnet object may possess both:

- a *stable identity root*, which names the continuing object, and
- an *evolving state root*, which changes as the object mutates.

For example, a DAO may remain the same DAO while its constitutional state root changes after amendments. The packet layer **MUST** allow clients to distinguish these.

3. Canonical Packet Schema

3.1. Packet philosophy

Packets are the transport-shaped faces of Kapnet acts. A packet is not identical to a durable object, but it is the mechanism by which objects are created, mutated, attested to, checkpointed, contracted, or retired.

Every packet **MUST** be:

- typed,
- hashable,
- serializable under a canonical order,

- attributable to a signer or signer-set,
- bounded in declared size,
- resolvable to a destination surface or namespace,
- replayable against an object state machine or relay policy.

3.2. Common packet envelope

All packet types share a common envelope. Packet-specific bodies extend this core.

Field	Type	Meaning
`version`	u16	protocol version
`packet_type`	u16	packet family discriminator
`packet_id`	bytes32	canonical digest of whole packet
`body_hash`	bytes32	digest of packet body only
`network_id`	string	primary anchoring network or tip family
`namespace`	string	destination namespace under ABNF grammar
`author_mode`	enum	single, threshold, delegated, provider, anonymous-set
`author_id`	bytes32 or string	signer root or representation root
`nonce`	u64 or bytes16	replay disambiguator
`timestamp`	u64	declared wall-clock or logical emission time
`base_anchor`	struct	optional base txid, block hash, height, or checkpoint relation
`priority_class`	enum	background, standard, expedited, contract, publication, knot
`fee_terms`	struct	postage, burn, bond, provider fee, or null
`weakwork_terms`	struct	difficulty target, nonce range, proof summary, or null
`capability_scope`	array	capabilities asserted or consumed
`preconditions`	array	object versions, parent references, required states
`references`	array	parent packets, cited objects, reply targets, proofs
`body_len`	u32	body length in bytes
`signature_envelope`	struct	one or more signatures plus scheme metadata
`receipt_request`	struct	desired service receipts
`retention_hint`	enum	ephemeral, retained, archival, canonical

3.3. Packet families

The core packet family registry in this volume is:

Type code	Name	Purpose
0x0001	SUBMIT	generic submission packet
0x0002	CREATE_OBJ	create room, guild, DAO, provider, archive, contract
0x0003	TRANSITION	state transition over existing object
0x0004	ATTEST	expert or role-bound attestation
0x0005	VOTE	vote or position under constitutional logic
0x0006	CONTRACT	open, amend, accept, terminate service contract
0x0007	RECEIPT	receipt or acknowledgment
0x0008	BRAID_ANNOUNCE	advertise braid region membership or candidate ordering
0x0009	KNOT_CLAIM	propose checkpoint over braid region / replay span
0x000A	FINGERPRINT	emit state digest for tip or domain
0x000B	RESOLUTION	dispute result, publication result, or governance settlement
0x000C	BUNDLE	bounded aggregate of packets for relay efficiency

3.4. Canonical serialization rules

The exact byte format may evolve, but conforming implementations **MUST** serialize packets under one deterministic rule set. This document adopts the following conceptual canonicalization constraints:

1. Field order is fixed by specification, not by producer preference.
2. Strings are UTF-8 and length-prefixed.
3. Arrays preserve explicit order if order is semantically meaningful; otherwise they are lexicographically sorted by canonical element digest.
4. Integers are unsigned big-endian unless explicitly marked signed.
5. Null and absent are not interchangeable; optional fields are explicitly tagged.
6. Whitespace in human-readable payloads is preserved in payload digests.
7. Signature envelopes are computed over the canonical envelope plus canonical body, excluding mutable transport wrappers.

Canonicalization rule of thumb

If two honest implementations serialize the same semantic packet to different byte strings, then the specification is insufficiently exact.

3.5. Size classes

To preserve bounded relay behavior, packets **SHOULD** declare one of the following size classes:

Class	Max bytes	Typical usage
Tiny	512	vote, receipt, short attestation, capability revoke
Small	4 KiB	ordinary submission, transition, simple contract action
Medium	32 KiB	constitution update, archive manifest, knot claim
Large	256 KiB	bounded bundles, publication manifests, composite proofs
Jumbo	implementation-gated	off-path retrieval object with manifest reference

Packets above the local relay acceptance threshold **MUST NOT** be silently truncated; they must be rejected or handled through a manifest-plus-fetch path.

4. Namespace Grammar and Destination Resolution

4.1. Design goals

The namespace grammar has three jobs:

1. distinguish tips, domains, and institutional surfaces,
2. make routing explicit under plurality,
3. keep names human-comprehensible without sacrificing parse discipline.

4.2. ABNF-like grammar

A conceptual grammar is given below. This is not final IETF ABNF syntax, but it is intentionally close.

```
namespace      = network ":" path
network        = netlabel / compound-network
compound-network = netlabel (":" netlabel)+
path           = segment / segment "." path
segment        = label / wildcard / typed-segment
label          = ALPHA *( ALPHA / DIGIT / "-" / "_" )
wildcard       = "*"
typed-segment  = kind "@" label
kind           = "room" / "guild" / "dao" / "fed" / "archive" / "provider" /
                 "claim" / "knot" / "pub" / "attest" / "index"
netlabel       = "BTC" / "BTX" / "BCK" / "KNX" / "CTB" / future-token
```

4.3. Examples

- `BTC:room@blz.history`
- `BTC:guild@blz.editorial.pub`
- `BTC:dao@kap.core.attest`
- `CTB:knot@newswire.daily`
- `BTC:BCK:claim@bridge.alpha`
- `BTC:provider@archivist.index`

4.4. Resolution order

When a packet names a destination namespace, a node **MUST** resolve it in the following order:

1. parse network prefix or compound prefix,
2. parse typed segments and validate against object registry,
3. locate destination object or wildcard policy rule,
4. validate sender capability against target surface,
5. evaluate local relay policy for admitted packet type and size class,
6. if unresolved, return deterministic `UNRESOLVED_NAMESPACE` failure rather than a generic drop.

4.5. Compound prefixes

Compound network prefixes represent cross-tip surfaces. For example, `BTC:BCK:claim@bridge.alpha` means the packet is primarily scoped to BTC-side routing but semantically concerns a BCK-related claim domain. Implementations **SHOULD** preserve the exact prefix sequence in packet digests because reordered meanings are not equivalent.

5. Identity, Delegation, and Representation

5.1. Author modes

The common envelope's `author_mode` defines how a packet's agency should be interpreted.

Mode	Meaning
`single`	one signer or one key-root authored the packet
`threshold`	a threshold or quorum of signers authored it
`delegated`	signer is acting under previously granted capability
`provider`	packet is authored by a service identity on behalf of a contract path
`anonymous_set`	packet discloses set-validity without stable personal identity
`federated`	packet is emitted from a federation-level signer root

5.2. Capability scopes

Capabilities **MUST** be explicit. A packet may claim capabilities such as:

- `submit.post`
- `submit.publish`
- `room.create`
- `guild.admit`
- `role.grant`
- `dao.amend`
- `attest.expert`
- `knot.propose`
- `provider.invoice`

Capability use without visible grant lineage **MUST** fail unless the target surface explicitly permits unauthenticated use.

5.3. Delegation packets

Delegation is modeled as ordinary state, not ambient implication. A delegation packet **MUST** define:

- grantor identity,
- grantee identity or role root,
- capability subset,
- destination scope,
- temporal bounds,

- revocation conditions,
- whether onward delegation is permitted.

6. Typed Bodies for Core Packet Families

6.1. SUBMIT body

The SUBMIT packet is the generic civil act. It is optimized for human-readable but machine-resolvable activity.

```
SUBMIT.body = {
  subject: string,
  content_type: enum(text/plain, text/markdown, app/json, app/kscript, app/mixed),
  content: bytes,
  tags: [string],
  visibility: enum(local, guild, public, contract, sealed),
  reply_to: optional packet_id,
  supersedes: optional packet_id,
  requested_actions: [action_ref],
  attachments: [attachment_ref]
}
```

6.2. CREATE_OBJ body

Used to instantiate rooms, guilds, DAOs, providers, archives, or federations.

```
CREATE_OBJ.body = {
  object_kind: enum(room, guild, dao, federation, provider, archive, contract),
  object_name: string,
  object_namespace: string,
  constitution_ref: optional hash_or_inline,
  initial_state: bytes,
  admin_root: bytes32,
  policy_flags: [flag],
  metadata: kv_map
}
```

6.3. TRANSITION body

The main packet for bounded object mutations.

```
TRANSITION.body = {
  target_object: object_id,
  prior_state_root: bytes32,
  transition_kind: enum(opcode_program, declarative_patch, lifecycle_change),
```

```
  transition_payload: bytes,
  expected_version: u64,
  merge_policy: enum(strict, replace, append, scoped-merge),
  failure_policy: enum(reject, no-op, escrow, dispute-open)
}
```

6.4. ATTEST body

```
ATTEST.body = {
  target_ref: object_or_packet_id,
  attestation_kind: enum(expert, editorial, archival, bridge, risk, audit),
  claims: [claim_blob],
  confidence: u8,
  expiry: optional u64,
  revocation_ref: optional packet_id
}
```

6.5. CONTRACT body

```
CONTRACT.body = {
  contract_kind: enum(relay, archive, index, publication, attestation, compute),
  action: enum(open, accept, amend, pause, resume, terminate, settle),
  provider_id: object_id,
  customer_id: object_id,
  service_terms: bytes,
  payment_terms: bytes,
  sla_terms: bytes,
  dispute_terms: bytes,
  settlement_refs: [base_or_overlay_ref]
}
```

6.6. RECEIPT body

```
RECEIPT.body = {
  receipt_kind: enum(admit, store, route, publish, index, checkpoint, reject),
  request_ref: packet_id,
  provider_id: object_id,
  status: enum(ok, partial, queued, failed),
  reason_code: string,
  evidence: bytes,
  timestamp_seen: u64,
  timestamp_final: optional u64
}
```

6.7. `FINGERPRINT` body

```
FINGERPRINT.body = {
  tip_label: string,
  height: u64,
  block_hash: bytes32,
  state_roots: [named_root],
  policy_profile: string,
  active_namespaces: [string],
  braid_head: optional bytes32,
  knot_head: optional bytes32,
  provider_digest: optional bytes32
}
```

7. Constitutional Opcode Core

7.1. Purpose

The opcode core is not a general-purpose VM. It is a bounded constitutional transition machine. Its task is to mutate object state under rules visible to replaying nodes.

7.2. Execution model

An opcode program executes against:

- a target object state,
- a capability set derived from author mode and grants,
- packet-local constants,
- optional immutable references,
- bounded memory slots and stack depth,
- deterministic failure rules.

7.3. State transition categories

The minimum viable transition categories are:

1. object lifecycle,
2. membership,

3. role and capability,
4. publication and archival,
5. constitution amendment,
6. dispute and resolution,
7. treasury or bonded resource state,
8. checkpoint governance.

7.4. Opcode table

Hex	Mnemonic	Class	Meaning
0x00	NOP	control	no-op
0x01	HALT	control	successful stop
0x02	ABORT	control	fail transition
0x10	NEW_OBJ	lifecycle	instantiate object shell
0x11	SET_META	lifecycle	set metadata key or blob
0x12	SET_STATE	lifecycle	replace state subtree
0x13	PATCH_STATE	lifecycle	patch path within state
0x14	ARCHIVE_OBJ	lifecycle	mark object archived
0x15	FREEZE_OBJ	lifecycle	freeze transition surface
0x20	ADD_MEMBER	membership	add member or member root
0x21	DROP_MEMBER	membership	remove member or role root
0x22	SUSPEND_MEMBER	membership	suspend member rights
0x23	RESTORE_MEMBER	membership	restore member rights
0x24	SET_ROLE	membership	bind role to member or set
0x25	CLEAR_ROLE	membership	remove role binding
0x30	GRANT_CAP	capability	grant explicit capability scope
0x31	REVOKE_CAP	capability	revoke capability
0x32	DELEGATE_CAP	capability	create delegated capability
0x33	EXPIRE_CAP	capability	expire grant by id
0x40	OPEN_ROOM	topology	create room under guild or federation
0x41	CLOSE_ROOM	topology	close room to new submissions
0x42	FORK_ROOM	topology	create descendant room with lineage
0x43	LINK_ARCHIVE	topology	attach archive to room/guild/object
0x50	AMEND_CONST	constitutional	amend constitution blob/root
0x51	SET_QUORUM	constitutional	change quorum formula
0x52	SET_POLICY	constitutional	change policy profile or flags
0x53	BIND_VALIDATOR	constitutional	bind validator or attestor class
0x54	UNBIND_VALIDATOR	constitutional	remove validator or attestor class
0x60	OPEN_DISPUTE	dispute	open dispute object
0x61	SET_DISPUTE_STATE	dispute	mutate dispute stage
0x62	RESOLVE_DISPUTE	dispute	close dispute with result
0x70	OPEN_CONTRACT	contract	instantiate service contract
0x71	AMEND_CONTRACT	contract	mutate service contract
0x72	TERMINATE_CONTRACT	contract	terminate contract

0x80	PUBLISH	publication	publish targeted object or archive entry
0x81	RETRACT	publication	retract or mark superseded
0x82	PIN_ARCHIVE	publication	request canonical retention
0x90	PROPOSE_KNOT	checkpoint	register knot candidate
0x91	ACCEPT_KNOT	checkpoint	accept knot in local constitutional surface
0x92	REJECT_KNOT	checkpoint	reject candidate with reason
0xA0	ASSERT_EQ	logic	assert equality
0xA1	ASSERT_HAS	logic	assert membership or capability
0xA2	ASSERT_STATE	logic	assert state path relation
0xA3	IF_FLAG	logic	condition on visible flag
0xA4	IF_ROLE	logic	condition on role root
0xB0	EMIT_RECEIPT	output	instruct provider receipt formation
0xB1	EMIT_EVENT	output	emit derived event packet

7.5. Execution bounds

To preserve bounded replay, conforming implementations **MUST** enforce at least:

- maximum opcode count per transition,
- maximum stack depth,
- maximum referenced objects per transition,
- maximum path length in state tree access,
- maximum emitted receipts or derived events.

An illustrative profile is:

Bound	**Illustrative cap**
Opcodes per program	256
Stack depth	32
Referenced objects	16
State path segments	32
Emitted derived events	8

7.6. Failure semantics

Transitions fail in one of four ways:

1. `STRUCTURAL_FAIL`: malformed packet or impossible field state,
2. `AUTH_FAIL`: insufficient capability or bad signature envelope,

3. `PRECONDITION_FAIL`: expected prior state or version not satisfied,
4. `EXEC_FAIL`: opcode program aborts or exceeds bounds.

Conforming implementations **MUST** return the same failure class for the same canonical input.

8. State Transition Rules by Object Class

8.1. Room transitions

Rooms **MUST** support at minimum the following lifecycle actions:

- create,
- open to submissions,
- restrict admission,
- archive,
- fork,
- rebind to archive,
- publish or retract canonical outputs.

A room transition **MUST NOT** alter its parent guild's constitution unless routed through the guild or DAO surface that governs such changes.

8.2. Guild transitions

Guilds **MUST** support:

- membership and role mutation,
- room creation or closure,
- policy and publication changes,
- archive attachment,
- optional treasury or provider relations.

8.3. DAO transitions

DAO transitions carry the highest constitutional weight. They **SHOULD** always name:

- prior constitutional root,
- expected version,
- authority mode,
- quorum evidence or references,
- exact amendment payload or program.

8.4. Provider transitions

Provider objects model a service actor. They **MUST** advertise at least:

- service classes,
- capacity profile,
- fee or work requirements,
- receipt kinds supported,
- retention and dispute policy,
- supported namespaces or object kinds.

9. Knot Claim Structure

9.1. Role of knot claims

A knot claim is the checkpoint packet by which a braid region or replay window is compressed into a candidate memory landmark. It does not supersede base chronology. It organizes overlay memory.

9.2. Knot claim body

```
KNOT_CLAIM.body = {
  knot_kind: enum(braid-window, archive-edition, publication-batch, governance-checkpoint
    ),
  target_namespace: string,
  braid_start_ref: packet_or_hash,
  braid_end_ref: packet_or_hash,
  ordered_member_refs: [packet_id],
```

```
  inclusion_root: bytes32,
  exclusion_root: optional bytes32,
  state_root_before: bytes32,
  state_root_after: bytes32,
  parent_knot: optional bytes32,
  mean_knot_time: optional u64,
  proposer_set: bytes32,
  attestation_refs: [packet_id],
  weakwork_summary: optional bytes,
  penalties_claimed: [penalty_claim],
  compression_manifest: bytes
}
```

9.3. Required invariants

A valid knot claim **MUST** satisfy:

1. every included member reference resolves to a visible and admissible packet,
2. the declared ordering is reproducible under the knot's ordering rule,
3. the claimed post-state root results from replaying the ordered member set against the declared prior state root,
4. the parent knot, if present, is consistent with the namespace and history relation,
5. all attestations and penalties referenced are themselves admissible.

9.4. Ordering rules

A namespace or constitution may choose one of several knot ordering rules:

- lexical packet-id order,
- weakwork-primary then lexical tie-break,
- relay-arrival window then lexical tie-break,
- constitution-defined weighted ordering.

The ordering rule **MUST** be declared by the relevant constitution or namespace policy before it is used by a claim.

9.5. Competing claims

Competing knot claims over the same braid span do not invalidate one another at the base layer. They create an overlay dispute surface. Implementations **SHOULD** expose competing claims visibly rather than flattening them into one invisible winner.

9.6. Knot acceptance workflow

A local node that receives a knot claim follows this sequence:

1. validate structure and signatures,
2. fetch missing members or mark claim incomplete,
3. replay ordered members against declared state root,
4. verify resulting state root and manifest digests,
5. evaluate namespace constitution for acceptance policy,
6. emit local acceptance, rejection, or pending status.

10. Fingerprints and Many-Tip State

10.1. Purpose of fingerprints

A fingerprint is not merely an informational dashboard item. It is the compact, routable surface by which nodes communicate what world they believe they are indexing.

10.2. Fingerprint fields

A robust fingerprint **SHOULD** include:

- tip label and network family,
- height and block hash,
- visible policy profile,
- key overlay state roots,
- active namespace digest,
- provider digest,
- braid and knot heads where relevant,
- timestamp of emission.

10.3. Cross-tip semantics

A node may maintain one fingerprint per tip:

$$\{F^{BTC}, F^{BTX}, F^{BCK}, F^{KNX}, F^{CTB}\}$$

Cross-tip surfaces **MAY** additionally expose composite fingerprints such as bridge domains, basket coordination zones, or publication domains that intentionally span several tips.

11. Relay Service Classes

11.1. Why service classes are explicit

Relays are not all the same. Formalizing service classes allows contracts, receipts, and user expectations to be aligned.

11.2. Core classes

Class	Meaning
Admit relay	validates syntax, signatures, and policy gates, then accepts into queue
Route relay	forwards packet to selected peers or providers
Retention relay	stores packet for contracted retention period
Archive relay	persists canonical or archival-grade copies
Index relay	builds searchable indices and content projections
Publication relay	services publication queues, editions, and canonical outputs
Knot relay	aggregates braid windows and supports checkpoint traffic
Bridge relay	specializes in multi-tip or compound-namespace surfaces

11.3. Admission policy structure

Each relay **MUST** be able to express admission policy as a tuple of:

- accepted namespaces,
- packet type allowlist,
- size-class limits,
- fee or weakwork minimums,
- capability or membership requirements,

- rate limits,
- retention defaults,
- receipt kinds emitted.

12. Relay Contract Templates

12.1. Template philosophy

A relay contract template is a named service shape, not merely an ad hoc blob. The goal is to make common service relationships legible and machine-checkable.

12.2. Template R1: Basic relay admission

Template R1 — Basic relay admission

Customer intent: obreakspace ordinary packet acceptance and forwarding.
Provider duties: validate envelope, queue packet, attempt forwarding to declared peer class, return admit receipt or reject receipt.
Customer duties: satisfy fee or weakwork minimum and packet shape rules.
SLA dimensions: queue latency, receipt latency, peer fanout floor.
Typical use: standard submissions and lightweight transitions.

12.3. Template R2: Retention contract

Template R2 — Retention contract

Customer intent: guarantee packet retention for a bounded period.
Provider duties: store packet or bundle, emit storage receipt, expose fetch endpoint, report expiration schedule.
Dispute basis: missing packet, corrupted digest, silent early deletion.
Typical use: institutional records, pre-publication archives, evidence trails.

12.4. Template R3: Archive pinning

Template R3 — Archive pinning

Customer intent: move a packet set or publication batch into canonical or near-canonical retention.
Provider duties: persist manifest, ensure retrievability, maintain integrity proof path, optionally mirror to peer archives.
Typical use: journal issue, governance checkpoint, final zine edition, historical record.

12.5. Template R4: Knot aggregation

Template R4 — Knot aggregation
Customer intent: obtain a provider's assistance in building, validating, or relaying candidate knot claims. **Provider duties:** collect braid members, apply declared ordering rule, emit candidate manifests, return accept or reject rationale. **Typical use:** publication windows, checkpoint-heavy DAO epochs, archive editions.

12.6. Template R5: Indexing contract

Template R5 — Indexing contract
Customer intent: searchable projection over packets, objects, or archives. **Provider duties:** maintain indexed views over declared namespaces, support query receipts or bounded query proofs, publish freshness indicators. **Typical use:** research guilds, editorial search, compliance or risk review.

13. Replay Pipeline

13.1. Pipeline stages

A conforming node's replay pipeline can be decomposed into eight stages:

1. ingest,
2. envelope validation,
3. namespace resolution,
4. capability and policy validation,
5. precondition check,
6. execution or state application,
7. receipt and index updates,
8. braid or checkpoint integration.

13.2. Replay pseudocode

```
for packet in ordered_visible_packets:
    if not valid_envelope(packet):
        reject(packet, STRUCTURAL_FAIL)
        continue
```

```
    target = resolve_namespace(packet.namespace)
    if not target:
        reject(packet, UNRESOLVED_NAMESPACE)
        continue
    if not auth_ok(packet, target):
        reject(packet, AUTH_FAIL)
        continue
    if not preconditions_ok(packet, target.state):
        reject(packet, PRECONDITION_FAIL)
        continue
    result = apply_packet(packet, target)
    if result.fail:
        reject(packet, result.code)
    else:
        commit(result.delta)
        emit_receipts(packet, result)
        update_braid_or_knot_views(packet, result)
```

13.3. Ordering sources

Nodes may encounter packets through multiple channels. Replay order therefore **MUST** be derived from an explicit ordering relation, not accidental storage order. Candidate order sources include:

- base-anchor order,
- knot-defined order,
- braid-local order,
- constitution-defined tie-breaks,
- packet-id lexical fallback.

14. Receipts

14.1. Why receipts matter

If relay and provider labor are to be economic and disputable rather than vague, service actions need receipts. A receipt is the machine-visible boundary between claim and delivered service.

14.2. Receipt taxonomy

Core receipt kinds are:

- `ADMIT`: packet accepted into provider queue,

- `STORE`: packet retained with digest proof,
- `ROUTE`: packet forwarded to declared peer class or endpoint set,
- `INDEX`: packet incorporated into indexed projection,
- `PUBLISH`: object or archive entry included in publication output,
- `KNOT`: knot candidate accepted, rejected, or queued,
- `REJECT`: structured failure with reason code.

14.3. Receipt reason codes

Reason codes **SHOULD** be namespaced. Illustrative examples:

- `auth.missing-capability`
- `policy.namespace-denied`
- `size.exceeds-class-limit`
- `work.below-threshold`
- `knot.replay-mismatch`
- `contract.sla-unavailable`

15. Node Workflows

15.1. Why workflows belong in the spec

A node specification that only describes packets and transitions, but never the actual operator loops those packets induce, remains incomplete. Human operators and advanced agents interact with systems through workflows. If the workflows are under-specified, the protocol becomes harder to build and easier to misuse.

15.2. Workflow W1: Join a guild

Goal: acquire visible membership and appropriate capability grants.

1. discover guild object and constitution root,
2. inspect namespace, policy profile, and admission rules,
3. create or select identity root,

4. emit admission request packet or contract-bound application packet,
5. satisfy required fees, bonds, weakwork, or attestations,
6. await admit, reject, or pending receipt,
7. if accepted, fetch membership state delta and capability set.

15.3. Workflow W2: Submit to a room

Goal: publish a structured submission into a bounded discussion surface.

1. resolve room namespace and target relay class,
2. compose `SUBMIT` packet with content and references,
3. attach fee terms or weakwork terms if required,
4. sign packet under available capability mode,
5. send to one or more relays,
6. collect admit or reject receipts,
7. observe inclusion into room braid and eventual archival or publication outcomes.

15.4. Workflow W3: Amend a DAO constitution

Goal: mutate constitutional state with bounded replay semantics.

1. fetch current DAO root and amendment procedure,
2. prepare `TRANSITION` with prior state root and opcode or declarative amendment payload,
3. attach quorum evidence or linked vote packets,
4. submit to constitutional relay and receipt collectors,
5. monitor dispute window or challenge period,
6. if accepted, update local state root and capability graph,
7. if contested, follow dispute workflow.

15.5. Workflow W4: Negotiate a relay contract

1. discover provider object and service classes,
2. compare fee, work, retention, and receipt terms,
3. emit `CONTRACT/open` packet,
4. receive provider accept, amend, or reject packet,
5. if accepted, bind provider id into local route policy,
6. monitor receipts and SLA indicators,
7. renew, pause, or terminate as needed.

15.6. Workflow W5: Propose a knot

1. select braid region and target namespace,
2. fetch all packet members and verify admissibility,
3. apply ordering rule,
4. replay against prior state root,
5. compute inclusion root, post-state root, and manifest,
6. emit `KNOT_CLAIM`,
7. gather attestations or provider receipts,
8. await local or contractual acceptance.

16. Node Interface Surfaces

16.1. Core operator panes

A serious Kapnet node cockpit **SHOULD** expose at least the following panes.

Pane	What it shows
Chain Monitor	per-tip height, block hash, sync state, fingerprint summary
Namespace Map	active namespaces, policy profiles, contract routes, unresolved destinations
Packet Queue	pending admits, rejects, retries, rate limits, weakwork classes
Replay Ledger	applied packets, failed packets, state deltas, version mismatches
Braidpool View	open braid windows, member counts, ordering rule, contention indicators
Knot Desk	candidate knots, acceptance status, replay mismatch alerts, attestation counts
Institution Browser	rooms, guilds, DAOs, federations, role graphs, constitution roots
Provider Console	contracts, SLAs, receipts, retention promises, index freshness
Dispute Board	active disputes, deadlines, evidence references, current stage
Archive Browser	pinned records, publication editions, manifests, retrieval status

16.2. Minimal screen layouts

A four-pane default layout is a plausible baseline:

1. top-left: chain and fingerprint monitor,
2. top-right: packet queue and receipt stream,
3. bottom-left: institution and namespace browser,
4. bottom-right: braid and knot desk.

16.3. Operator alarms

The interface **SHOULD** surface at least these alarms:

- fingerprint divergence across peers,
- unresolved namespace growth,
- contract SLA breaches,
- repeated auth failures,
- precondition mismatches during replay,
- competing knot claims over the same span,
- archive integrity failure,
- cross-tip bridge surface inconsistency.

17. Worked Examples

17.1. Example 1: Editorial room post

A BLZ editorial room post may be serialized as:

```
version=1
packet_type=SUBMIT
network_id=BTC
namespace=BTC:room@blz.editorial
priority_class=standard
author_mode=delegated
author_id=member_root_77
capability_scope=[submit.post]
preconditions=[room.version=12]
body.subject="Draft cover idea"
body.content_type=text/markdown
body.content="Need review on masthead spacing and caption tone."
body.tags=[draft, cover, review]
receipt_request=[ADMIT, STORE]
```

17.2. Example 2: Guild membership admission

```
packet_type=TRANSITION
namespace=BTC:guild@blz
capability_scope=[guild.admit]
preconditions=[guild.version=44, role.editor.exists]
transition_payload=ADD_MEMBER(candidate_root=0xabc..., role=contributor)
receipt_request=[ADMIT, STORE, INDEX]
```

17.3. Example 3: Knot claim over publication batch

```
packet_type=KNOT_CLAIM
namespace=BTC:pub@blz.issue07
knot_kind=publication-batch
braid_start_ref=packet_1001
braid_end_ref=packet_1089
ordered_member_refs=[...]
state_root_before=0x111...
state_root_after=0x222...
inclusion_root=0x333...
parent_knot=0xprev...
attestation_refs=[editorial_attest_1, archive_attest_2]
receipt_request=[ADMIT, KNOT, STORE]
```

18. Security and Failure Surfaces

18.1. Replay ambiguity

If canonical serialization is underspecified, two honest nodes may compute different packet ids or state roots. This is catastrophic for replay discipline. Canonicalization defects therefore deserve the same seriousness as signature defects.

18.2. Delegation leakage

Poorly scoped delegation can create ambient authority. Implementations **SHOULD** expose a human-readable explanation of why a packet is authorized, not merely a boolean pass or fail.

18.3. Namespace capture

Human-readable namespaces can be captured, spoofed, or shadowed. Object registries and typed segments mitigate but do not abolish this risk. Good clients should distinguish display names from canonical object ids.

18.4. Provider overcentralization

If too many civil functions route through a tiny provider class, Kapnet's formal decentralization may coexist with practical chokepoints. Volume III cannot solve this, but the contract and receipt model at least makes concentration visible.

18.5. Knot ossification or manipulation

Checkpoint politics can create social lock-in or strategic censorship pressure. Visible competing claims, replay proofs, and policy-declared ordering rules reduce but do not eliminate this risk.

19. Application and Use-Case Mapping

19.1. Why map applications in a technical volume

A protocol is better understood when its packet families and object models are pressed against actual use. Here the use-case atlas is not decorative. It demonstrates that the exact specification is broad enough to host many domains without dissolving into vagueness.

19.2. Editorial and publishing systems

Kapnet can model editorial rooms, issue planning, peer review, publication queues, archive pinning, and knot-based edition checkpoints. Receipts matter because publication and archival labor are services, not magic.

19.3. Research and scientific guilds

Research guilds can use attestation packets, expert role grants, archive manifests, reproducibility disputes, and canonical issue or report knots. Cross-tip namespaces can distinguish archival anchoring from publication or discussion domains.

19.4. Firms and cooperatives

Firms can model membership, delegation, project rooms, expense or contract approvals, provider relationships, publication of official decisions, and audit archives. The opcode core is sufficient for many bounded organizational state transitions long before one needs a general smart-contract machine.

19.5. Media networks and investigative collaboration

Investigative guilds benefit from sealed visibility classes, staged publication contracts, archive pinning, attestation surfaces, and controlled release knots. Provider classes matter especially here because retention, secrecy, and publication timing are all service questions.

19.6. Open policy and municipal governance

Rooms can represent working groups, guilds can represent departments or assemblies, DAOs can define amendment and quorum logic, and publication knots can checkpoint ordinances, minutes, or policy bulletins. The state machine remains bounded and replayable.

19.7. Cross-tip bridge communities

Compound namespaces and fingerprints support communities or providers that intentionally span several tips without forcing those tips into one false identity. This is where forkgivening becomes operational rather than rhetorical.

19.8. Agentic organizations

AI-assisted or agent-heavy institutions need explicit capability scopes, bounded execution, receipt-rich provider markets, and replayable dispute surfaces. Kapnet's discipline is particularly valuable

here because autonomous or semi-autonomous agents magnify the danger of implicit authority and invisible workflows.

20. Implementation Sequence

20.1. Phase 1: packet and object registry

Implement:

- common packet envelope,
- typed packet ids,
- namespace parser,
- object registry,
- signature envelope verifier,
- receipt skeletons.

20.2. Phase 2: replay engine

Implement:

- precondition checks,
- deterministic state application,
- opcode runtime with hard bounds,
- reason-coded failures,
- braid-local ordering views.

20.3. Phase 3: providers and contracts

Implement:

- provider objects,
- contract templates,
- receipts,
- archive and index services,
- local SLA dashboards.

20.4. Phase 4: knots and many-tip awareness

Implement:

- knot claims,
- replay proof path,
- competing claim visibility,
- per-tip fingerprints,
- compound namespace routes,
- bridge relay surfaces.

20.5. Phase 5: operator cockpit

Implement the pane model and end-to-end workflows so users can actually see the system they are operating.

21. Conclusion

This volume tries to do one thing above all: remove excuses for vagueness. It does so by forcing the doctrine down into exact surfaces: envelopes, fields, kinds, rules, transitions, receipts, contracts, fingerprints, and workflows.

That descent changes the conversation. One no longer speaks of submissions in a purely poetic sense, because packet families and body schemas now exist. One no longer speaks of DAO constitutionality only as a metaphor, because an opcode core and precondition rules now exist. One no longer invokes knots as a suggestive image only, because knot envelopes, ordering rules, and acceptance flows now exist. One no longer says relays are important without precision, because service classes, templates, receipts, and SLA surfaces now exist. One no longer imagines the node cockpit as a mood board, because panes, alarms, and operator loops have been decomposed.

The result is still not the final irreversible standard. Important choices remain open: exact cryptographic suites, precise binary encoding, optimized state storage, transport protocols, privacy envelopes, and proof systems. But the field of admissible choices is now narrower. Implementations built after this point would not merely be inspired by Kapnet; they would be forced to either conform to or visibly depart from a concrete shape.

That is the correct next stage for a system that wants to be more than a cloud of compelling ideas.

A. Quick Reference: Common Failure Codes

Code	Meaning
`STRUCTURAL_FAIL`	malformed envelope or impossible packet shape
`AUTH_FAIL`	signatures or capability chain invalid
`UNRESOLVED_NAMESPACE`	destination could not be resolved
`PRECONDITION_FAIL`	expected object state or version mismatch
`EXEC_FAIL`	opcode execution aborted or exceeded bounds
`POLICY_DENY`	relay or provider policy denied admission
`SIZE_DENY`	packet exceeds size class limit
`WORK_DENY`	weakwork or pricing terms insufficient
`KNOT_MISMATCH`	replayed state root differs from claimed post-state

B. Quick Reference: Capability Examples

- `submit.post`
- `submit.publish`
- `room.create`
- `room.close`
- `guild.admit`
- `guild.suspend`
- `role.grant`
- `role.revoke`
- `dao.amend`
- `attest.expert`
- `contract.open`
- `contract.amend`
- `knot.propose`
- `knot.accept`
- `archive.pin`
- `provider.invoice`

C. One-Page Build Checklist

1. Parser for common envelope and type registry.
2. Canonical serialization test vectors.
3. Namespace parser and resolver.
4. Signature and delegation verifier.
5. Object state store and versioned roots.
6. Opcode interpreter with hard execution bounds.
7. Receipts and reason-coded failures.

8. Provider and contract store.
9. Knot claim replay checker.
10. Operator cockpit with pane-based state visibility.